SUPMTI

École Supérieure de Management,
de Télécommunications et d'Informatique

Projet d'evaluation .Net :

Application de suivi de séries TV

Réalisé par :

ImaneAzza
YoussefLamkhantar

Encadrant par :

Prof. Mounir Omari

Master ISI — 1ère année | 2025-2026

Plan de la Présentation

1. Introduction & Contexte
2. Cahier des Charges
3. Conception
4. Architecture Technique
5. Stack Technologique
6. Réalisation & Tests
7. Conclusion

Contexte du Projet

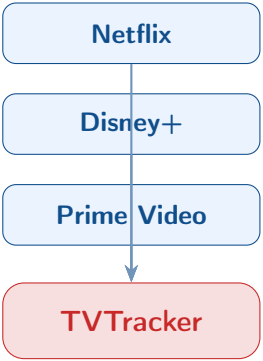
Problématique

Les utilisateurs consomment de nombreuses séries sur plusieurs plateformes. La gestion manuelle de leur progression devient **complexe et difficile**.

Solution proposée — TVTracker

Une application web centralisée pour :

- Gérer les séries regardées
- Suivre la progression épisode par épisode
- Noter et commenter les contenus visionnés
- Obtenir des statistiques personnalisées



Étude de l'Existant

Critère	TV Time	Hobi
Type	Plateforme web	Application mobile
Fonctions	Suivi + recommandations + communauté	Interface simple, séries/films
Limites	Pas d'API REST moderne	Pas de version web complète

Notre valeur ajoutée

Architecture découplée **ASP.NET Core** + **React** + sécurité **JWT** + déploiement **Docker**

Besoins Fonctionnels

Visiteur (non connecté)

- S'inscrire / Se connecter
- Parcourir le catalogue
- Voir le détail d'une série

Utilisateur authentifié

- Suivre une série (statuts)
- Marquer des épisodes comme vus
- Noter et commenter
- Gérer ses favoris
- Consulter son tableau de bord

En cours **Terminé** **Abandonné** **Planifié**

Besoins Non Fonctionnels & Contraintes

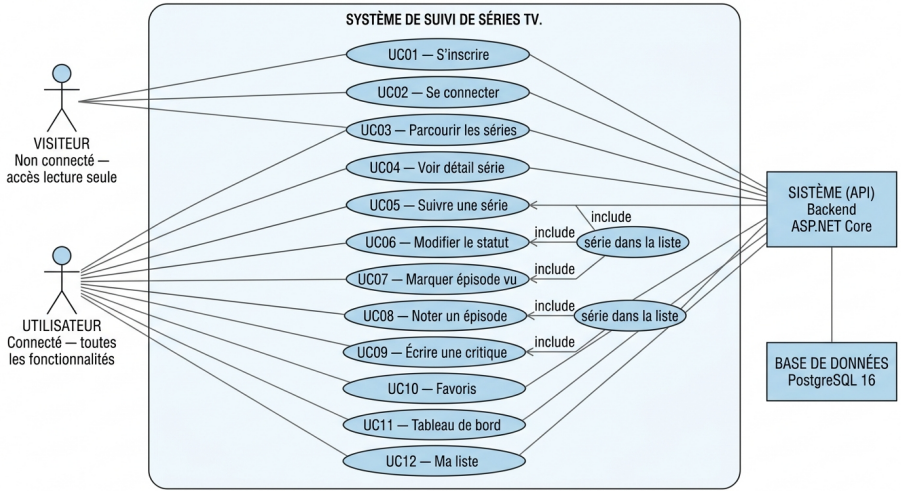
Qualité système

- **Sécurité** : authentification JWT
- **Performance** : temps de réponse rapide
- **Compatibilité** : multi-navigateurs
- **Scalabilité** : architecture évolutive
- **Ergonomie** : interface intuitive

Contraintes du projet

- **Délai** : livraison avant le 24 avril
- **Stack** : ASP.NET Core + React + PostgreSQL
- **Déploiement** : Docker (cross-platform)
- **Données** : intégration API externes
- **Planification** : Diagramme de Gantt

Diagramme de Cas d'Utilisation



Diagrammes de Séquence

1. Authentification

1. Formulaire React
2. `POST /api/auth/register`
3. Validation données
4. Hash BCrypt
5. INSERT utilisateur
6. Token JWT généré
7. Retour 200 `OK`

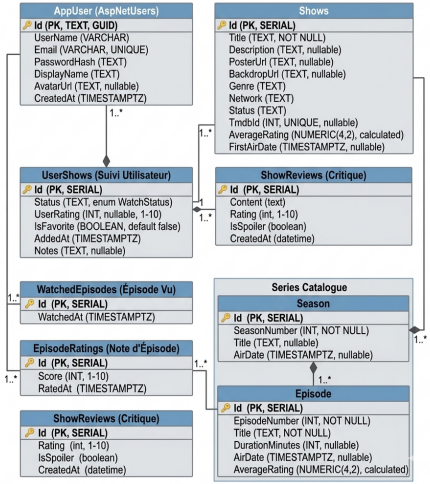
2. Suivi d'une série

1. Clic "Suivre"
2. `POST /shows/{id}/track`
3. Validation JWT
4. Vérification UserShows
5. INSERT UserShows
6. Retour 200 `OK`

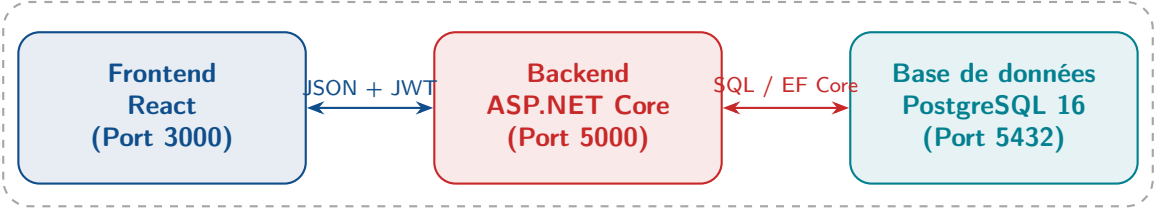
3. Épisode vu

1. Clic "Marquer vu"
2. `POST /episodes/{id}/watched`
3. Validation JWT
4. INSERT WatchedEpisodes
5. MAJ progression
6. Retour 200 `OK`

Modèle Conceptuel de Données



Architecture Globale



Conteneurisation Docker Compose — Port 80

Architecture Frontend

Présentation (UI)

React Router : navigation entre pages
Pages : Home, Browse, MyList, Profile
Composants réutilisables :

- ShowCard, StarRating
- Navbar, Layout, Formulaires

Gestion d'état

- **Zustand** : état global (auth, token, profil)
- **React Query** : cache et synchronisation serveur

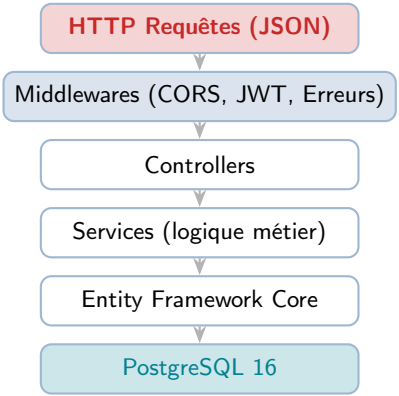
Client HTTP & Sécurité

Axios + Intercepteur JWT
En-tête : `Authorization: Bearer`

Stack Frontend

React + **TypeScript** + Vite
React Router v6
Zustand + React Query
Axios + Tailwind CSS

Architecture Backend



DTOs principaux

`RegisterDto`, `LoginDto`,
`ShowDto`, `EpisodeDto`,
`UserShowDto`, `StatsDto`

Sécurité

- **ASP.NET Identity** (hash BCrypt)
- **JWT** 7 jours ; Claims : Id, Email, DisplayName
- Migrations automatiques au démarrage

Docker Compose

nginx + dotnet/aspnet :8.0 + postgres :16

Technologies Utilisées

Backend

ASP.NET Core 8

Entity Framework Core

ASP.NET Identity + JWT

C#

Frontend

React + TypeScript

React Router v6

Zustand + React Query

Axios + Tailwind CSS

PostgreSQL 16

Docker Compose

Figma (UI/UX)

Implémentation

Backend

- Architecture REST multi-couches
- Controllers → Services → Repository
- Authentification JWT sécurisée
- Migrations EF Core automatiques
- API externes (données séries)

Frontend

- Composants réutilisables React
- State management Zustand
- Cache serveur React Query
- Intercepteur Axios JWT
- Interface conçue sur Figma

Base de données PostgreSQL 16

`AppUsers`, `Shows`, `Seasons`, `Episodes`, `UserShows`, `WatchedEpisodes`, `ShowReviews`, `EpisodeRatings`

Déploiement Docker Compose

- `nginx` (frontend, port 80)
- `aspnet:8.0` (API, port 5000)
- `postgres:16` (DB, port 5432)

Tests & Validation

Types de tests réalisés :

1. **Tests API REST** — vérification des endpoints
2. **Tests d'authentification** — inscription, connexion, JWT
3. **Tests fonctionnels** — gestion séries et épisodes
4. **Tests d'intégration** — frontend $\leftrightarrow$ backend
5. **Validation finale** — scénarios d'usage réels

Résultats

- Toutes les API répondent correctement
- Authentification JWT validée
- Progression épisodes fonctionnelle
- Interface fluide et réactive

Conclusion & Perspectives

Bilan du projet

- Application full-stack complète et fonctionnelle
- Architecture moderne, sécurisée et scalable
- Interface ergonomique conçue sur Figma
- Déploiement facilité par Docker
- Compétences renforcées en développement web

Perspectives

- Recommandations intelligentes (ML)
- Application mobile (React Native)
- Fonctionnalités sociales (amis, partage)
- Notifications de nouveaux épisodes
- Intégration TMDB avancée

Merci pour votre attention

Réalisé par :

ImaneAzza
YoussefLamkhantar

Encadrant par :

Prof. Mounir Omari

Master ISI — 1ère année | 2025-2026